DrainGuard AI

A Flood and Drainage Risk Dashboard for Lagos

WFEO ECBAP AI + Engineering Challenge 2026: Water, Sanitation and Resilient Infrastructure

Hammed Yusuf Akinteye
Concept Originator and Product Lead
hammedyusuf802@gmail.com

Sanusi Mustapha Babansoro
Lead Engineer, Full-Stack Development
sanusimustapha387@gmail.com

August 19, 2026

What This Document Is

This is a full walkthrough of DrainGuard AI: the problem it addresses, exactly how it is built, every piece of AI and decision-making logic inside it, what data is genuinely real versus simulated, and where its current limits are. It is written so a non-technical reader and a technical reviewer can both follow it.

Contents

1 The Problem **2**

2 What DrainGuard AI Is **2**

3 System Architecture **2**

4 End-to-End Data Flow **3**

5 The AI and Decision-Intelligence Layer **3**
5.1 Multi-Criteria Weighted Risk Scoring 3
5.2 Real-Time Data Fusion 4
5.3 Rule-Based Classification: the Risk Drivers Panel 4
5.4 Explainable by Design 4
5.5 Domain-Informed Stochastic Simulation 4
5.6 A Designed Upgrade Path to Machine Learning 4
5.7 A Secondary Heuristic: Predicted Overflow 5

6 What's Real, and What's Simulated **5**

7 Technology Stack **6**

8 Known Limitations and Roadmap **6**

9 Why This Fits the Challenge **6**

10 Glossary **7**

1 The Problem

Lagos floods often, and a major cause is drainage failure: drains that are blocked, undersized, or simply overwhelmed by heavy rain. Right now, nobody has an easy, live way to see which drains, in which neighborhoods, are closest to failing. Flooding usually becomes visible only once water is already in the street, and by then it is too late to act preventively. DrainGuard AI exists to close that gap: it turns scattered, real-time signals (rain, soil, drain condition) into one clear, constantly updating picture of risk, before the flood happens rather than after.

2 What DrainGuard AI Is

DrainGuard AI is a live web dashboard, branded internally as **DIPS** (Drainage Infrastructure Prioritization System), covering ten real, named flood-prone locations across Lagos. It shows:

- An overall city-wide flood risk level (Low, Moderate, or High) as a gauge, with a plain-language explanation of why that level was reached.
- A live map with a colour-coded pin at every monitored drain, labelled with its real place name.
- A sortable table of every drain's water level, blockage percentage, computed risk score, and a recommended action.
- Live rainfall, temperature, weather condition, and soil moisture for Lagos, refreshed automatically every 7 minutes.
- A 24-point rainfall trend chart and a feed of the most urgent recent alerts.

Nothing on the page requires a login, and nothing refreshes manually. The whole dashboard is a live subscription that updates itself as new data arrives.

3 System Architecture

Frontend: Next.js (App Router) with React and Tailwind CSS. All dashboard components are client components subscribed directly to the backend, so any change in the database is pushed to the browser the moment it happens. There is no polling and no manual refresh in the intended design.

Backend: Convex, a backend-as-a-service that bundles four things DrainGuard AI needs into one system:

1. A real-time database (`drains`, `readings`, `rainfallSnapshots` tables).
2. **Queries**: read-only functions the frontend subscribes to (for example `getDashboard`), which re-run and re-push automatically whenever the underlying data changes.
3. **Mutations**: functions that write to the database, such as inserting a new sensor reading.
4. **Actions and cron jobs**: functions that can call external services (like a weather API) and can be scheduled to run automatically, independent of anyone having a browser or terminal open.

Scheduled jobs (`convex/crons.ts`): two cron jobs fire every 7 minutes. One fetches live weather from Open-Meteo; the other generates the next simulated sensor reading for each drain. Because these run on Convex's cloud infrastructure, not on a developer's laptop, they keep firing whether or not anyone has the project open.

No authentication layer: the dashboard is intentionally public and read-only for now, which kept the build free of unnecessary complexity for an MVP timeline.

4 End-to-End Data Flow

1. **Every 7 minutes**, the `refreshRainfallSnapshot` cron calls Open-Meteo's free public API for Lagos and pulls the 24-hour rainfall total, current temperature, a weather condition code, and topsoil moisture. This is saved as a new `rainfallSnapshots` row.

2. **In the same cycle**, the `generateReadings` cron reads each drain's most recent water level and blockage percentage, and computes the next value for each (see Section 6 for exactly how), tagged `source: "simulated"`.

3. The `getDashboard` query, the single place all business logic lives, combines the latest rainfall snapshot with every drain's latest reading, runs the DIPS risk-scoring formula on each one, and returns one aggregated object: per-drain risk, an overall city score, counts of High, Moderate, and Low drains, and the Risk Drivers breakdown.

4. Because the frontend calls this query through `useQuery` (a live subscription, not a one-off fetch), Convex pushes the new result to every open browser tab automatically the instant the underlying data changes. No refresh is needed.

5. Every visual element on the dashboard, the gauge, the map pins, the table rows, the alerts feed, and the rainfall chart, reads from this one query result. There is exactly one source of truth; nothing is calculated twice or drifts out of sync.

5 The AI and Decision-Intelligence Layer

Honest framing first. DrainGuard AI does not use a trained machine-learning model (no neural network, no model trained on historical flood data) in its current form, because no historical Lagos drain-sensor dataset exists yet to train one on. What it uses instead is a transparent, rule-based decision-intelligence system, which is a legitimate and long-standing branch of AI. The field of expert systems and decision support systems predates deep learning and is still widely used in engineering and infrastructure contexts precisely because it is explainable. The sections below break down exactly which AI and decision-science techniques are used, and how they combine.

5.1 Multi-Criteria Weighted Risk Scoring

This is the core of DIPS. Instead of judging flood risk from a single number, it combines three independent signals into one 0 to 100 score, each contributing a capped share:

$$\begin{aligned}\text{Score} = {} & 40 \times \min\left(\frac{\text{water level (cm)}}{150},\ 1\right) \\ & + 30 \times \min\left(\frac{\text{blockage \%}}{100},\ 1\right) \\ & + 30 \times \min\left(\frac{\text{rainfall 24h (mm)}}{50},\ 1\right)\end{aligned}$$

Water level contributes up to 40 points, blockage up to 30 points, and rainfall up to 30 points. The result is classified: 70 or above is High, 40 or above is Moderate, and anything lower is Low.

This is a form of **Multi-Criteria Decision Analysis (MCDA)**, a well-established AI and operations-research technique for turning several unlike measurements into one ranked decision,

used in fields from engineering risk assessment to medical triage. The weights (40, 30, 30) reflect an engineering judgement call that water level is the most direct flood indicator, with blockage and rainfall as roughly equal contributing factors. This is documented in the code as an illustrative starting point, not a value calibrated against a real Lagos drain-capacity survey.

5.2 Real-Time Data Fusion

The dashboard does not just display raw numbers side by side. It fuses independently sourced live signals, weather-model data from Open-Meteo, simulated drain telemetry, and soil moisture, into a single coherent risk assessment per location, and into one overall city score. Combining multiple heterogeneous, asynchronously updating data streams into one consistent judgement is the same underlying principle used in sensor fusion systems in robotics and autonomous vehicles, applied here to civil infrastructure monitoring instead of physical sensors on a car.

5.3 Rule-Based Classification: the Risk Drivers Panel

Alongside the single DIPS score, the dashboard breaks risk down into individual Risk Drivers: Rainfall Forecast, Drain Blockage, Water Levels, and, when real soil data is available, Soil Saturation. Each is independently classified Low, Moderate, or High using the same threshold logic (ratio of 0.7 or above is High, 0.4 or above is Moderate, otherwise Low) applied to that specific signal. This is deliberate: a single overall score tells you that something is risky, but a rule-based breakdown tells you why, which is what an engineer or emergency planner actually needs to act on. Soil Saturation is only shown when a real reading exists that cycle. Tide Level was considered and intentionally left out entirely, because no free, reliable live tide feed for Lagos was found; a fabricated "Moderate" badge would have been dishonest, so the feature was omitted rather than faked.

5.4 Explainable by Design

A guiding decision in building DrainGuard AI was to make every number traceable to its inputs. This follows the principle of Explainable AI (XAI): rather than a black-box model that outputs a risk percentage with no way to inspect why, every score here can be decomposed back into its three named components, and every component back into a live data source. This matters especially for a civil-engineering audience who need to trust and act on a system's output, not just observe it.

5.5 Domain-Informed Stochastic Simulation

Because no physical IoT flood sensors are installed anywhere in Lagos at this scale yet, true of essentially every city and not a Lagos-specific gap, water level and blockage readings are generated by a calibrated random-walk simulation rather than read from hardware. This is not random noise: each new reading is nudged from the previous one, pulled upward in proportion to how much it has actually rained in the last 24 hours (real data), and pulled back down during dry periods to mimic natural drainage. This keeps the simulated telemetry behaviourally plausible, since it responds to real weather the way a real drain would, while every simulated row is explicitly tagged `source: "simulated"` in the database and surfaced as such in the interface, so it is never presented as live hardware.

5.6 A Designed Upgrade Path to Machine Learning

The architecture was deliberately built so a trained ML model can be dropped in later without a redesign. `calculateRiskScore()` is the single function that turns inputs into a score. Swapping its rule-based formula for a trained classifier, for example a gradient-boosted model or a small neural network trained on historical flood-incident data once such a dataset exists, only requires replacing that one function; the rest of the pipeline (data fusion, storage, live delivery, explainability panel)

stays unchanged. The feature set the system already computes and stores, water level, blockage percentage, 24-hour rainfall, and soil moisture, is exactly the feature set a real model would need as input, so today's data pipeline is already ML-ready even though today's decision logic is rule-based.

5.7 A Secondary Heuristic: Predicted Overflow

The table's Predicted Overflow (24h) column is a simple derived heuristic: 0.72 times the DIPS score plus 0.24 times the blockage percentage, clamped to a range of 2 to 99 percent. It is explicitly a computed estimate for prioritisation display, not a hydrological forecast or a claim about true overflow probability, documented as such in the code so it is never mistaken for something more rigorous than it is.

6 What's Real, and What's Simulated

Being explicit about this is the whole point of building an honest MVP rather than a demo that only looks finished.

- **Genuinely real:** the 10 drain locations (actual named Lagos flood-prone neighbourhoods with real coordinates); all rainfall, temperature, weather-condition, and soil-moisture data (live from Open-Meteo, refreshed every 7 minutes); and every risk-scoring calculation applied to that live data.
- **Simulated, but honestly labelled and weather-responsive:** water level and blockage percentage per drain. This is because no physical sensor exists yet, not because of a shortcut. Every simulated reading is tagged in the database and never displayed as if it came from hardware.
- **Not yet wired into the interface:** a community-reporting feature (`submitCommunityReport`) already exists on the backend, validated server-side, ready to accept a resident's report of an observed flooded drain as a genuine, zero-cost, sensor-free real-data channel. The submission form on the frontend is still being built. This is an openly acknowledged loose end, not a hidden gap.

No value on the dashboard is hardcoded to look more finished or more real than it is. Wherever a genuinely reliable, free live data source was not available, such as tide level or verified live flood-incident history, the corresponding feature was removed entirely rather than faked.

7 Technology Stack

Layer	Tool	Why
Frontend framework	Next.js (React)	Modern, fast, free to host; App Router client components pair naturally with live data subscriptions.
Styling	Tailwind CSS	Rapid, consistent UI styling without hand-written CSS files.
Backend / database	Convex	Combines a real-time database, serverless functions, and cron scheduling in one free-tier service, with no separate hosting, queue, or scheduler needed.
Weather data	Open-Meteo API	Free, no API key, provides real rainfall, temperature, weather code, and soil moisture for any coordinates.
Map	Leaflet with CARTO tiles	Free, open-source interactive mapping with a dark basemap matching the dashboard's visual design.
Charts	Recharts	Lightweight, React-native charting for the rainfall trend.
Hosting	Vercel	Free tier, automatic deployment on every GitHub push, zero server management.

8 Known Limitations and Roadmap

Framed openly as an MVP, a minimum viable product, not a finished product:

- Several sidebar navigation items (Map View, Drain Monitor, DIPS Ranking, Maintenance, Reports, IoT Devices, Data History, Settings) are currently visual chrome only. They do not show fabricated data, but they are not functional yet.
- Risk-score weights and thresholds (150cm drain capacity, 50mm heavy-rain reference, the 0.7 and 0.4 classification cutoffs) are documented, reasonable placeholders, not values calibrated against a Lagos-specific engineering survey.
- The community-report submission form is not yet live in the interface; the backend is ready and the frontend is pending.
- No live tide data source is wired in; the feature was omitted rather than faked.
- The project runs on Convex's free development deployment rather than a separately provisioned production deployment. This is functionally fine for an MVP, but worth noting for anyone evaluating production-readiness.

9 Why This Fits the Challenge

The WFEO ECBAP AI + Engineering Challenge's Water, Sanitation and Resilient Infrastructure track is looking for practical, deployable engineering thinking, not just an impressive-sounding algorithm. DrainGuard AI's core argument is that honest, explainable decision intelligence, built on real live weather data, transparent scoring, and a clearly labelled path from simulation to real sensors, is a more responsible and more genuinely useful submission than a system that quietly fabricates data to look more finished than it is. It is a working MVP, a minimum viable product, not a mock-up or a slide deck, with a clear, documented path to becoming a full deployed system as real sensor data becomes available.

10 Glossary

Term	Plain-English meaning
API	A way for one piece of software to ask another for data. Here, DrainGuard asks Open-Meteo's API for weather.
Cron job	A task scheduled to run automatically and repeatedly (here, every 7 minutes), with nobody needing to trigger it by hand.
Query (live/reactive)	A request for data that stays open. When the underlying data changes, the result is pushed to the screen automatically.
MVP	Minimum Viable Product, the smallest honest version of a system that actually works, before every feature is finished.
Expert system	An AI system that makes decisions using explicit, human-written rules, rather than a model learned automatically from data.
MCDA	Multi-Criteria Decision Analysis: combining several different measurements into one ranked decision.
XAI	Explainable AI: AI systems designed so a person can see and understand why a particular output was produced.
Sensor fusion	Combining data from multiple different sources into one unified, more reliable picture.
Stochastic simulation	A model that generates plausible values using controlled randomness, rather than either being fixed or being pure noise.

In One Sentence

DrainGuard AI turns live Lagos weather data into a real-time, explainable flood-risk picture for ten real drainage hotspots, using transparent multi-criteria decision logic and honestly labelled simulation, deliberately built with a clear upgrade path to real sensors and a trained model as that data becomes available.

Prepared for the WFEO ECBAP AI + Engineering Challenge 2026 submission.